

Used ChatGPT 5.2
Prompts and answers below

You said:

explain step by step how to scrape data from the wayback machine. specifically <https://gasprices.aaa.com/?state=FL> the individual gas prices for individual counties on a smallest unit (daily) for years

ChatGPT said:

Thought for 47s

Below is the exact workflow I'd use to scrape **Florida county gas prices** from the **Wayback Machine** for <https://gasprices.aaa.com/?state=FL>, plus the "gotchas" that will make or break your pipeline.

One big reality check up front: **Wayback usually does *not* have truly daily data** for years. It only has pages for the days/times it archived them. AAA says county averages are updated daily, but that doesn't mean Wayback captured them daily.

0) What you're actually scraping (the key decision)

On AAA's Florida page, the "county retail prices" content is presented as a visualization/interactive element, while the HTML we can easily read includes metro blocks (Daytona, Miami, etc.) and headline state averages.

So you have **two possible data sources**:

1. **Best case (what you want): an underlying JSON/API request** that returns *county* → *price* (for the map).
2. **Fallback:** scrape whatever is actually embedded in archived HTML (often metro averages, not counties).

Your goal ("county, daily, for years") basically requires #1.

1) Identify the county data endpoint (do this on the live site)

You do this once, then your Wayback scraper becomes straightforward.

Steps (Chrome)

1. Open `https://gasprices.aaa.com/?state=FL`
2. Right-click → **Inspect**
3. Go to **Network**
4. Filter by **Fetch/XHR**
5. Hard refresh the page (**Cmd+Shift+R** / **Ctrl+Shift+R**)
6. Click around / hover the county map if it's interactive.
7. Look for requests whose response looks like JSON and contains county-ish keys (names, FIPS, etc.)

What you're hunting for:

- A URL that returns something like:
 - `{ "county": "Broward", "regular": 2.91, ... }, ... ]`
 - or a keyed object by county code

Once you have that URL, copy it.

Why this matters: Wayback exposes archived content via **CDX** (index of captures) and **Memento** (fetch a capture).

2) Use the Wayback CDX API to list archived snapshots

Wayback has two layers you'll use:

- **CDX API**: "what timestamps exist for this URL?"
- **Archived fetch**: "give me the content at timestamp T"

(These are the two standards mentioned in the Wayback docs: CDX + Memento.)

For the *page* URL

You'll query CDX for:

- `https://gasprices.aaa.com/?state=FL`

For the *county JSON endpoint* (ideal)

You'll also query CDX for:

- whatever endpoint you discovered in step 1

Important: If the endpoint itself wasn't archived, your scrape can fail even if the HTML page was archived (because the HTML expects live JS/data).

3) Convert "available captures" into "one capture per day"

Because Wayback captures are irregular, you usually:

- build a set of timestamps
- group them by date
- pick the "best" timestamp per date (e.g., closest to noon, or latest capture that day)

This gives you a pseudo-daily series *for the days archived*.

4) Fetch the archived content correctly (raw mode)

When you fetch archived pages, use the "id_" form so you get **raw content** without Wayback rewriting as much:

- **Archived page** pattern:

- `https://web.archive.org/web/{TIMESTAMP}id_/https://gasprices.aaa.com/?state=FL`

Same idea for an API endpoint:

- `https://web.archive.org/web/{TIMESTAMP}id_/{ENDPOINT_URL}`

5) Parse the response into a tidy table

- If it's JSON: parse JSON → DataFrame
- If it's HTML: parse with BeautifulSoup and extract the relevant nodes
- Add `date` from the timestamp
- Save incremental results to CSV/Parquet so you don't lose progress

Concrete Python template (the backbone)

This is a working skeleton you can adapt once you know the county endpoint.

```
import requests
import pandas as pd
from datetime import datetime

CDX = "https://web.archive.org/cdx/search/cdx"

def cdx_list(url: str, from_year=None, to_year=None, limit=None):
    params = {
        "url": url,
        "output": "json",
```

```

        "fl": "timestamp,original,statuscode,mimetype",
        "filter": "statuscode:200",
        "collapse": "timestamp:8", # collapse to YYYYMMDD (1 capture/day, roughly)
    }

    if from_year:
        params["from"] = str(from_year)
    if to_year:
        params["to"] = str(to_year)
    if limit:
        params["limit"] = str(limit)

    r = requests.get(CDX, params=params, timeout=30)
    r.raise_for_status()
    data = r.json()
    header, rows = data[0], data[1:]
    return pd.DataFrame(rows, columns=header)

def wayback_fetch(timestamp: str, original_url: str) -> str:
    # raw capture
    wb_url = f"https://web.archive.org/web/{timestamp}id_{original_url}"
    r = requests.get(wb_url, timeout=30)
    r.raise_for_status()
    return r.text

def ts_to_date(timestamp: str) -> str:
    # timestamp like 20220115123456
    dt = datetime.strptime(timestamp[:8], "%Y%m%d")
    return dt.date().isoformat()

# 1) List captures for the Florida page (example)
page_url = "https://gasprices.aaa.com/?state=FL"

```

```

caps = cdx_list(page_url, from_year=2015, to_year=2025)
print(caps.head())

# 2) If you find a county JSON endpoint, do the same:
# endpoint_url = "PASTE_THE_REAL_ENDPOINT_YOU_FOUND"
# caps_api = cdx_list(endpoint_url, from_year=2015, to_year=2025)
# print(caps_api.head())

# 3) Fetch + parse example (JSON endpoint version)
def parse_county_json(text: str) -> pd.DataFrame:
    # depends on actual structure—adjust once you see it
    obj = requests.models.complexjson.loads(text)
    return pd.DataFrame(obj)

# Example loop (endpoint version)
# out = []
# for _, row in caps_api.iterrows():
#     ts = row["timestamp"]
#     raw = wayback_fetch(ts, endpoint_url)
#     df = parse_county_json(raw)
#     df["date"] = ts_to_date(ts)
#     out.append(df)
# result = pd.concat(out, ignore_index=True)
# result.to_csv("aaa_fl_county_wayback.csv", index=False)

```

Notes on that code

- `collapse=timestamp:8` is a quick way to reduce to ~1 capture/day (not perfect, but practical).
 - If you want “closest capture to noon,” don’t collapse; instead group by date and choose your own timestamp.
-

6) The two big failure modes (and how to handle them)

Failure mode A: “Wayback archived the HTML, but not the county endpoint”

Fixes:

- Try scraping county values from the archived HTML (if embedded anywhere).
- Or accept that you can only get metro/state from HTML captures.
- Or broaden your data source (EIA / other historical datasets) if your project truly needs complete daily coverage.

Failure mode B: “The county map is rendered client-side and not present in the HTML”

This is common. The HTML you see in an archive may not include the county table at all—because the JS that would fetch it doesn’t run (or the endpoint wasn’t captured).

That’s why step **1 (finding the endpoint)** is the critical move.

What I can already confirm from the AAA page itself

- AAA explicitly frames county averages as “updated daily.”
 - The Florida page contains metro averages and statewide headline averages in plain HTML.
-

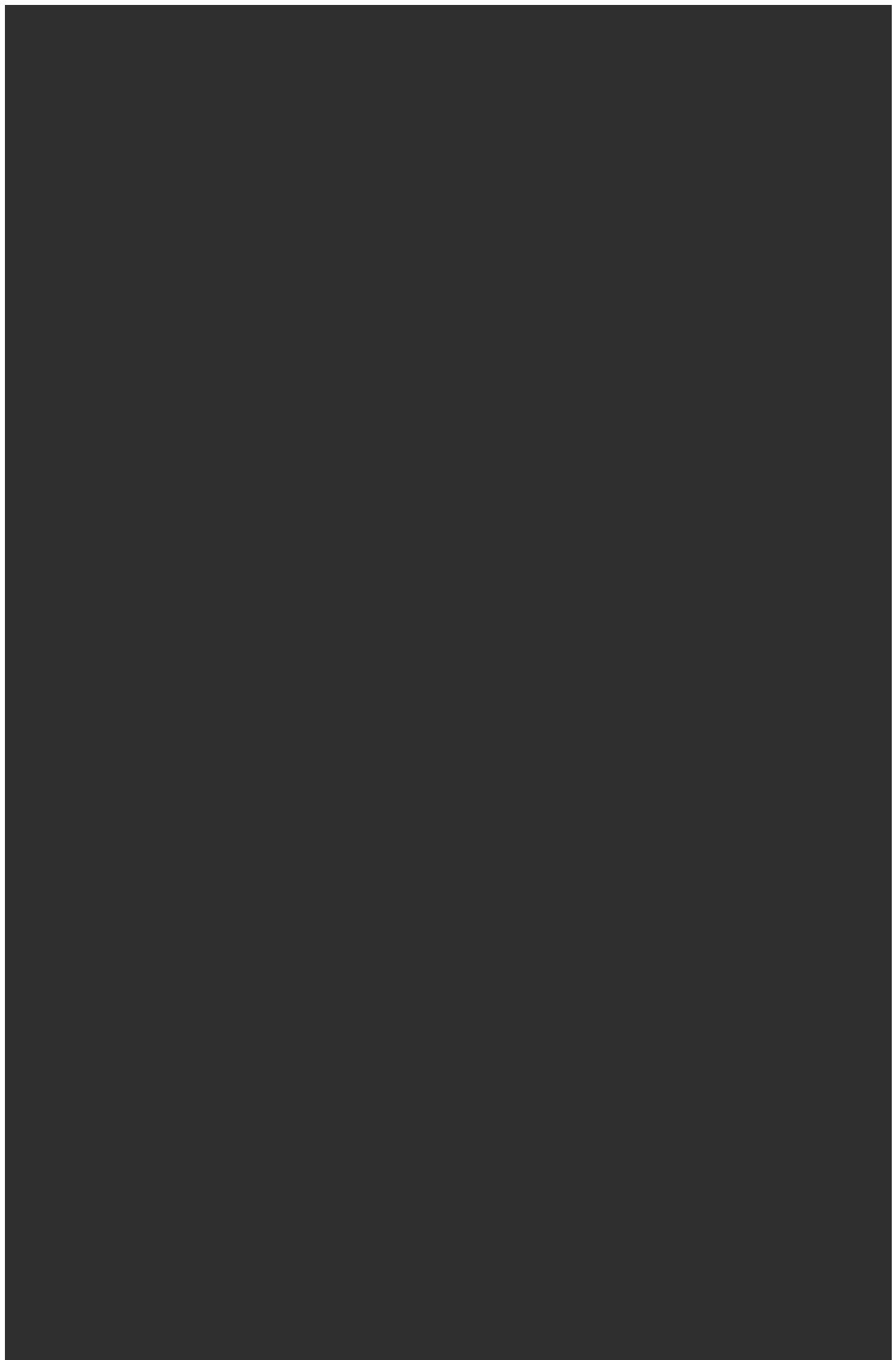
If you want the “smallest unit (daily) for years”

Be prepared to re-scope to: “**daily for archived days**” unless you find a separate AAA historical endpoint that returns time series (many sites don’t expose this publicly).

If you paste the **one Network/XHR URL** you find for the county data (step 1), I can tell you exactly:

- whether Wayback has captures for it,
- what the JSON structure looks like,
- and how to parse it into a clean `date, county, price` panel.

You said:



i cant find the JSON endpoint. it is metros through, not counties like i had thought. this is todays page, you click the dropdown. how can i find the real url?

ChatGPT said:

Thought for 14s

If DevTools shows **no XHR/Fetch** when you expand a metro, that usually means one of these is true:

1. **The “real data” is already embedded in the HTML** (server-rendered), and the dropdown just shows/hides it.
2. It’s embedded inside a **script tag** as a big JS object (not fetched live).
3. It is fetched, but **not as JSON** (could be HTML fragment, GraphQL POST, or a WordPress REST call you didn’t notice).

Here’s the step-by-step way to figure out which one you’re dealing with and get the “real URL.”

Step 1 — Prove whether it’s embedded or fetched

A) Quick “view source” test (fastest)

1. Load the page: <https://gasprices.aaa.com/?state=FL>
2. Right click → **View Page Source** (NOT Inspect)
3. Use find (Cmd/Ctrl+F) for a metro name you see (e.g., Fort Lauderdale)

If you find the numbers (\$2.908, etc.) in the page source, then there’s no JSON endpoint you’re missing — the data is in the HTML and your “real URL” is just the page itself (plus different states/metros).

If you do NOT find them, then it’s likely inserted by JS, and you need Step 2.

Step 2 — If it's JS-inserted, find the payload (even if it isn't "JSON")

A) Network tab setup that actually catches it

1. Open DevTools → **Network**
2. Turn on:
 - **Preserve log**
 - **Disable cache**
3. Filter to:
 - **Fetch/XHR** AND also check **Doc** and **JS** (seriously—lots of "data" comes down with the doc or a JS bundle)
4. Hard refresh: Cmd/Ctrl + Shift + R
5. Now click the dropdown / expand metros

Then scan for requests that look like:

- wp-json (WordPress REST)
- graphql
- api
- or any request whose **Response** contains your metro name

B) If you still see "nothing"... it may be bundled into a JS file

1. In Network → click the largest JS file(s) (often main*.js / app*.js)
2. Open the **Response** tab
3. Cmd/Ctrl+F for Fort Lauderdale or Daytona

If you find metro names inside a bundle, the data might be baked in, or the bundle contains the URL pattern for where data is fetched.

Step 3 — Check the DOM: the "real URL" may be hiding in plain sight

In your screenshot there are arrows / dropdown UI. Often those are actual links or contain IDs.

A) Inspect the metro header element

1. Right-click "Fort Lauderdale" row → **Inspect**
2. Look for:
 - an `<a href=...">` (link to a metro page)
 - `data-*` attributes (e.g., `data-metro="fort-lauderdale"` or an ID)
 - `aria-controls=..."` pointing to a hidden panel

B) If it's an `<a href=...`

That href is your "real URL" for that metro. Open it in a new tab and repeat Step 1 (view source find numbers).

Many sites do "state page → metro page", where the metro page is the stable unit you'd scrape historically.

Step 4 — Use the "Search" tool inside DevTools (this is the sleeper move)

If you can't spot it in Network:

1. DevTools → **Sources**
2. Press Cmd/Ctrl + Shift + F (global search)
3. Search for a unique string:
 - Fort Lauderdale (or any metro name)
 - or Yesterday Avg
 - or one of the exact prices like 2.908 (if it's current and stable)

This will show you *where* that text exists:

- in the HTML doc
- in a JS bundle
- in an inline script (common for WordPress sites)

If you find it in an inline script, look around it for a big object like:

- `window.__INITIAL_STATE__ = {...}`
- `__NEXT_DATA__ = {...}`
- or a JSON blob inside a `<script type="application/json">`

That blob is basically “the endpoint response,” just embedded.

Step 5 — If it’s embedded HTML (most likely given your symptoms), accept the simplest scrape strategy

If Step 1 finds the values in page source, then the “real URL” is:

- State page: `https://gasprices.aaa.com/?state=FL`

And your scraper should:

- download the HTML
- parse each metro block + its table
- store a row per (date, metro, fuel_grade, current/yesterday/week/month/year)

This is totally scrapeable from Wayback too (as long as Wayback captured the page).

The “daily for years” limitation still applies: you’ll only get dates Wayback archived.

Step 6 — A sanity check: you might be expecting counties, but AAA is giving metros only

Given what you’re seeing, AAA Florida appears to be reporting by **metro areas** (not county), and each metro has a small comparison table (current/yesterday/week/month/year). That means:

- There may *not* be a county dataset on that site at all.
- If your project needs county-level, you'll likely need a different data source (or change the unit to metro).

The fastest way to unblock you

Do this and paste back what you see:

1. On the Florida page, **View Page Source** and search for `Fort Lauderdale`.
 - If you find it: tell me yes/no and whether the prices are visible there.
2. When you Inspect the "Fort Lauderdale" header row, tell me:
 - Is it an `<a href="...">`? If yes, paste that href.
 - If not, do you see any `data-` attributes?

With just those two facts, I can tell you definitively whether:

- there's a hidden endpoint you missed, or
- you should scrape the HTML directly (and how to target the exact tags reliably).

You said:

```
yes, it is embedded <td></td> <td>$5.719</td> <td>6/11/22</td> </tr> </tbody>
</table></div> </div> </div> <h3 data-cost="2.9080000" data-title>Fort
Lauderdale</h3> <div> <div class="tblwrap"> <table class="table-mob"> <thead>
<tr> <th></th> <th>Regular</th> <th>Mid</th> <th>Premium</th>
<th>Diesel</th> </tr> </thead> <tbody>
```

ChatGPT said:

Perfect — if it's embedded like that, you **don't need an endpoint**. Your "real URL" is simply the Wayback-captured HTML for:

- `https://gasprices.aaa.com/?state=FL`

...and you scrape the metro blocks straight out of the HTML each capture.

The snippet you pasted tells us two key things:

- Each metro name is in an `<h3>` like:
`<h3 data-cost="2.9080000" ...>Fort Lauderdale</h3>`
- The prices are in the **table immediately after** that `<h3>`.

So the plan becomes:

1. Use **CDX API** to list archived captures of that URL
2. For each capture timestamp:
 - download the archived HTML (`.../{timestamp}id_/...`)
 - parse metros + table rows
 - record `capture_date` + `metro` + `row_label` (Current Avg, Yesterday Avg, etc.) + fuel prices
3. Save to CSV / parquet

Below is a clean, reliable way to do that.

Step 1 — Get one Wayback capture list (timestamps)

Use CDX to get snapshots. You can either:

- keep **all** captures and later choose one per day, or
- let CDX collapse roughly to 1/day.

I recommend **not collapsing at first** (more control), then you dedupe yourself.

CDX request idea (conceptually):

- URL: `https://web.archive.org/cdx/search/cdx`
- params:
 - `url=https://gasprices.aaa.com/?state=FL`
 - `output=json`
 - `fl=timestamp,statuscode`

- `filter=statuscode:200`
-

Step 2 — Download the archived HTML (raw form)

For each timestamp, fetch:

```
https://web.archive.org/web/{TIMESTAMP}id_/https://gasprices.aaa.com/?state=FL
```

The `id_` matters: it gives you cleaner raw HTML.

Step 3 — Parse the metro blocks from HTML (robustly)

What to target

You'll parse a repeating structure:

- `<h3 ...>METRO NAME</h3>`
- next sibling `<div> ... <table> ... </table> ... </div>`

Inside the table:

- header row: Regular, Mid, Premium, Diesel
- body rows: Current Avg, Yesterday Avg, Week Ago Avg, Month Ago Avg, Year Ago Avg
- each value like `$2.908`

Practical parsing approach

Use BeautifulSoup:

- iterate over all `h3` tags that look like metro titles
 - for each:
 - grab the next `<table>` after it
 - read its rows into records
-

Step 4 — Put it all together (working Python)

```
import re
import time
import requests
import pandas as pd
from bs4 import BeautifulSoup
from datetime import datetime

CDX = "https://web.archive.org/cdx/search/cdx"
STATE_URL = "https://gasprices.aaa.com/?state=FL"

def cdx_timestamps(url: str, from_year=None, to_year=None, limit=None) -> list[str]:
    params = {
        "url": url,
        "output": "json",
        "fl": "timestamp,statuscode",
        "filter": "statuscode:200",
    }
    if from_year:
        params["from"] = str(from_year)
    if to_year:
        params["to"] = str(to_year)
    if limit:
```

```

    params["limit"] = str(limit)

    r = requests.get(CDX, params=params, timeout=30)
    r.raise_for_status()
    data = r.json()
    rows = data[1:]
    return [row[0] for row in rows]

def wayback_html(timestamp: str, original_url: str) -> str:
    wb_url = f"https://web.archive.org/web/{timestamp}id_/ {original_url}"
    r = requests.get(wb_url, timeout=30)
    r.raise_for_status()
    return r.text

_money = re.compile(r"^\$?\s*([0-9]+(?:\.[0-9]+)?)\s*$")

def parse_price(cell_text: str):
    t = cell_text.strip()
    if not t or t == "—" or t.lower() == "n/a":
        return None
    m = _money.match(t.replace(",", ""))
    return float(m.group(1)) if m else None

def ts_to_date(timestamp: str) -> str:
    # use the capture date (not necessarily "current avg date")
    dt = datetime.strptime(timestamp[:8], "%Y%m%d")
    return dt.date().isoformat()

def parse_metros(html: str, capture_date: str) -> pd.DataFrame:
    soup = BeautifulSoup(html, "html.parser")
    records = []

```

```
# AAA uses h3 for each metro section (based on your snippet)
for h3 in soup.find_all("h3"):
    metro = h3.get_text(strip=True)
    if not metro:
        continue

    # Heuristic: metros are usually not generic headings
    # and often have data-cost attribute per your snippet
    if not h3.has_attr("data-cost"):
        continue

    table = h3.find_next("table")
    if not table:
        continue

    # Read header to map columns (Regular, Mid, Premium, Diesel)
    thead = table.find("thead")
    headers = []
    if thead:
        ths = thead.find_all("th")
        headers = [th.get_text(strip=True) for th in ths]

    # Expected: ["", "Regular", "Mid", "Premium", "Diesel"]
    # We'll hard-map by position to avoid weird header quirks
    fuel_cols = ["regular", "mid", "premium", "diesel"]

    tbody = table.find("tbody")
    if not tbody:
        continue
```

```

for tr in tbody.find_all("tr"):
    tds = tr.find_all(["td", "th"])
    if len(tds) < 2:
        continue

    row_label = tds[0].get_text(strip=True) # e.g., "Current Avg."
    values = [parse_price(td.get_text()) for td in tds[1:1+len(fuel_cols)]]

    rec = {
        "capture_date": capture_date,
        "metro": metro,
        "row_label": row_label,
    }
    for col, val in zip(fuel_cols, values):
        rec[col] = val

    records.append(rec)

return pd.DataFrame(records)

def scrape_wayback_fl(from_year=2015, to_year=2025, sleep_s=0.5, max_caps=None):
    ts = cdx_timestamps(STATE_URL, from_year=from_year, to_year=to_year, limit=max_caps)
    all_frames = []

    for i, tstamp in enumerate(ts, 1):
        cdate = ts_to_date(tstamp)
        try:
            html = wayback_html(tstamp, STATE_URL)
            df = parse_metros(html, cdate)
            df["wayback_timestamp"] = tstamp
            all_frames.append(df)

```

```

except Exception as e:
    print(f"Failed {tstamp}: {e}")

if sleep_s:
    time.sleep(sleep_s)

if i % 50 == 0:
    print(f"Processed {i}/{len(ts)} captures...")

if not all_frames:
    return pd.DataFrame()

out = pd.concat(all_frames, ignore_index=True)

# Optional: dedupe to one record per (date, metro, row_label) using latest capture that date
out = out.sort_values(["capture_date", "metro", "row_label", "wayback_timestamp"])
out = out.drop_duplicates(["capture_date", "metro", "row_label"], keep="last")

return out

# Example run:
# df = scrape_wayback_fl(from_year=2018, to_year=2022, sleep_s=0.2, max_caps=500)
# df.to_csv("aaa_fl_metro_wayback.csv", index=False)

```

Step 5 — Turn this into “daily-ish” metro series

Remember: you’ll only have **days Wayback captured**. Your dedupe step makes “one per day” using the latest capture.

If you want *regular gasoline only* and *current avg only*, filter:

- `row_label == "Current Avg."`
 - `keep regular`
-

Two important details (so you don't get wrecked later)

1) "6/11/22" in your snippet

That date you saw (6/11/22) might be part of some other table (often a "historical high/low" block) or a different section. Don't assume it's the capture date. For time series, **use the Wayback timestamp date** as the observation date unless the site explicitly labels the date for the "current avg."

2) The units are metros, not counties

If your hurricane design needs county-level, you'll either:

- accept metro-level as your geography, or
 - switch data source.
-

What I need from you (no extra hunting, just 1 thing)

Paste **10–20 lines above** the `<h3 data-cost="2.9080000"...>` and **10–20 lines below** the start of that `<table>` (just like you did). I want to confirm:

- whether the `<h3>` always has `data-cost`
- whether the `<table>` class is consistent (`table-mob`)
- what the row labels look like exactly (punctuation varies)

Then I can tighten the parser so it doesn't miss edge cases (and so you don't scrape garbage silently).

You said:

```
<td>$3.551</td> </tr> <tr> <td>Month Ago Avg.</td> <td>$2.858</td>
<td>$3.288</td> <td>$3.630</td> <td>$3.552</td> </tr> <tr> <td>Year Ago
Avg.</td> <td>$3.006</td> <td>$3.403</td> <td>$3.744</td> <td>$3.668</td>
</tr> </tbody> </table> </div> <div class="row-sm"> <h4>highest recorded average
price</h4> <div class="tblwrap"> <table class="record-table"> <thead> <tr>
<th></th> <th></th> <th></th> <th>Price</th> <th>Date</th> </tr> </thead>
<tbody> <tr> <td>Regular Unleaded</td> <td></td> <td></td> <td>$4.895</td>
<td>6/13/22</td> </tr> <tr> <td>Diesel</td> <td></td> <td></td>
<td>$5.719</td> <td>6/11/22</td> </tr> </tbody> </table> </div> </div> </div>
<h3 data-cost="2.9080000" data-title>Fort Lauderdale</h3> <div> <div
class="tblwrap"> <table class="table-mob"> <thead> <tr> <th></th>
<th>Regular</th> <th>Mid</th> <th>Premium</th> <th>Diesel</th> </tr>
</thead> <tbody> <tr> <td>Current Avg.</td> <td>$2.908</td> <td>$3.393</td>
<td>$3.685</td> <td>$3.552</td> </tr> <tr> <td>Yesterday Avg.</td>
<td>$2.920</td> <td>$3.389</td> <td>$3.680</td> <td>$3.569</td> </tr> <tr>
<td>Week Ago Avg.</td> <td>$2.867</td> <td>$3.348</td> <td>$3.643</td>
<td>$3.545</td> </tr> <tr> <td>Month Ago Avg.</td> <td>$2.862</td>
<td>$3.334</td> <td>$3.647</td> <td>$3.517</td>
```

ChatGPT said:

This is exactly what we needed. ✓ The structure is consistent and scrapeable, and you've now got **two distinct table types per metro block**:

- **Main metro table:** `<table class="table-mob">` with rows like *Current Avg.*, *Yesterday Avg.*, *Week Ago Avg.*, *Month Ago Avg.*, *Year Ago Avg.*
- **Record table:** `<table class="record-table">` with rows like *Regular Unleaded* and *Diesel* and their *highest recorded average price* + date.

So the scraper should explicitly grab **only** `table.table-mob` for your time series, and optionally also grab `table.record-table` as metadata.

Below is the tightened, reliable parsing logic (no guessing, no fragile assumptions).

What the HTML guarantees (based on your snippet)

For each metro:

1. Metro header:

```
<h3 data-cost="2.9080000" data-title>Fort Lauderdale</h3>
```

2. Next sibling `<div>` contains:

- a `<table class="table-mob">` (what you want)
- optionally a `<table class="record-table">` (what you probably don't want for daily series)

So the rule is:

For each `h3[data-cost]`, parse the *first* `table.table-mob` that follows it.

That will **avoid accidentally scraping** the “highest recorded average price” table.

Updated parsing code (drop-in replacement)

```
import re
import requests
import pandas as pd
from bs4 import BeautifulSoup
from datetime import datetime

_money = re.compile(r"^\$?\s*([0-9]+(?:\.[0-9]+)?)\s*$")

def parse_price(text: str):
    t = (text or "").strip()
```

```

if not t or t in {"—", "-"}:
    return None

m = _money.match(t.replace(",", ""))
return float(m.group(1)) if m else None

```

```

def ts_to_date(timestamp: str) -> str:
    return datetime.strptime(timestamp[:8], "%Y%m%d").date().isoformat()

```

```

def parse_metro_tables(html: str, capture_date: str, wayback_timestamp: str) -> pd.DataFrame:

```

```

    soup = BeautifulSoup(html, "html.parser")
    rows = []

```

```

    for h3 in soup.select("h3[data-cost]"):
        metro = h3.get_text(strip=True)
        if not metro:
            continue

```

```

    # key: ONLY the "table-mob" table after this metro header
    table = h3.find_next("table", class_="table-mob")
    if table is None:
        continue

```

```

    # rows look like: [label, regular, mid, premium, diesel]
    for tr in table.select("tbody tr"):
        cells = [c.get_text(strip=True) for c in tr.find_all("td")]
        if len(cells) < 2:
            continue

```

```

        label = cells[0] # "Current Avg." etc.
        vals = cells[1:5] # regular, mid, premium, diesel

```

```
rec = {
    "capture_date": capture_date,
    "wayback_timestamp": wayback_timestamp,
    "metro": metro,
    "label": label,
    "regular": parse_price(vals[0]) if len(vals) > 0 else None,
    "mid": parse_price(vals[1]) if len(vals) > 1 else None,
    "premium": parse_price(vals[2]) if len(vals) > 2 else None,
    "diesel": parse_price(vals[3]) if len(vals) > 3 else None,
}
rows.append(rec)

return pd.DataFrame(rows)
```

If you also want the “highest recorded average price” table (optional)

That table is **not time-varying daily**, so you probably don’t need it for hurricane effects. But if you want it:

- It’s `table.record-table`
- It contains “Price” and “Date” columns.

I can give you that parser too, but I’d only include it if you have a reason.

One more critical “tell it like it is” point

Even with this clean HTML scrape, you still won't get "daily for years" unless Wayback captured it daily.

Your dataset will be:

"metro prices on dates the page was archived"

That can still be totally usable for a project (especially around hurricane windows), but it's not a complete daily panel.

Next step: connect this to Wayback capture collection

If you want, paste 5–10 sample Wayback timestamps (or tell me the year range you care about most, like 2016–2022), and I'll give you the full end-to-end script that:

- pulls CDX timestamps
- samples to 1 per day
- scrapes all metros
- outputs a clean CSV like:

```
date, metro, label, regular, mid, premium, diesel
```

No fluff, just working.

You said:

```
https://web.archive.org/web/20211130124705/https://gasprices.aaa.com/?state=FL or 2022-2025
```

ChatGPT said:

Thought for 16s

Awesome — if it's embedded, you're in the cleanest-case scenario: **no JSON endpoint exists** because the page is basically already a dataset.

Your Wayback URL example works, but for scraping you'll want the **raw** version (no toolbar/rewrites as much):

- Toolbar version (what you pasted):
`https://web.archive.org/web/20211130124705/https://gasprices.aaa.com/?state=FL`
- **Raw HTML** version (better for parsing):
`https://web.archive.org/web/20211130124705id_/https://gasprices.aaa.com/?state=FL`

Now here's the end-to-end way to scrape **2022–2025**.

1) Get all archived timestamps for 2022–2025 (CDX API)

CDX is the Wayback "index of captures."

CDX query (copy/paste into browser)

```
https://web.archive.org/cdx/search/cdx?url=https://gasprices.aaa.com/?state=FL&from=2022&to=2025&output=json&fl=timestamp,statuscode&filter=statuscode:200
```

What you get back is JSON with a header row, then many timestamps.

Reality check: there will be **multiple captures per day** sometimes, and **many missing days**.

2) Decide how you want "daily"

You've got two sane options:

Option A (recommended): "one capture per day"

Group captures by `YYYYMMDD` and pick the **latest** timestamp that day.

Option B: keep everything

Useful if you want intraday changes, but it explodes your data volume.

I'll implement Option A below.

3) Scrape the HTML for each capture

Parsing rule (based on your snippet)

For each metro block:

- metro name: `h3[data-cost]`
 - data table: the next `table.table-mob`
 - ignore `table.record-table` unless you explicitly want the "highest recorded" info
-

4) Full working Python script

```
import re
import time
import requests
import pandas as pd
from bs4 import BeautifulSoup
from datetime import datetime

STATE_URL = "https://gasprices.aaa.com/?state=FL"
CDX = "https://web.archive.org/cdx/search/cdx"
```

```
# ---- helpers ----  
_money = re.compile(r"^\$?\s*([0-9]+(?:\.[0-9]+)?)\s*$")
```

```
def parse_price(text: str):  
    t = (text or "").strip()  
    if not t or t in {"—", "-"}:  
        return None  
    m = _money.match(t.replace(",", ""))  
    return float(m.group(1)) if m else None
```

```
def cdx_timestamps_2022_2025():  
    params = {  
        "url": STATE_URL,  
        "from": "2022",  
        "to": "2025",  
        "output": "json",  
        "fl": "timestamp,statuscode",  
        "filter": "statuscode:200",  
    }  
    r = requests.get(CDX, params=params, timeout=30)  
    r.raise_for_status()  
    data = r.json()  
    rows = data[1:] # first row is header  
    return [row[0] for row in rows] # timestamps
```

```
def choose_one_per_day(timestamps):  
    # pick the latest capture per YYYYMMDD  
    by_day = {}  
    for ts in timestamps:  
        day = ts[:8]  
        if day not in by_day or ts > by_day[day]:
```

```

        by_day[day] = ts
    return [by_day[d] for d in sorted(by_day.keys())]

def wayback_raw_html(timestamp: str):
    # raw capture (less toolbar / rewriting)
    url = f"https://web.archive.org/web/{timestamp}id_/{STATE_URL}"
    r = requests.get(url, timeout=30)
    r.raise_for_status()
    return r.text

def capture_date_from_ts(timestamp: str) -> str:
    return datetime.strptime(timestamp[:8], "%Y%m%d").date().isoformat()

def parse_metro_tables(html: str, capture_date: str, wayback_timestamp: str) -> pd.DataFrame:
    soup = BeautifulSoup(html, "html.parser")
    out = []

    for h3 in soup.select("h3[data-cost]"):
        metro = h3.get_text(strip=True)

        if not metro:
            continue

        table = h3.find_next("table", class_="table-mob")

        if table is None:
            continue

        for tr in table.select("tbody tr"):
            cells = [c.get_text(strip=True) for c in tr.find_all("td")]

            if len(cells) < 2:
                continue

```

```

label = cells[0] # Current Avg., Yesterday Avg., etc.
vals = cells[1:5] # Regular, Mid, Premium, Diesel

out.append({
    "date": capture_date,
    "wayback_timestamp": wayback_timestamp,
    "metro": metro,
    "label": label,
    "regular": parse_price(vals[0]) if len(vals) > 0 else None,
    "mid": parse_price(vals[1]) if len(vals) > 1 else None,
    "premium": parse_price(vals[2]) if len(vals) > 2 else None,
    "diesel": parse_price(vals[3]) if len(vals) > 3 else None,
})

return pd.DataFrame(out)

def run_scrape(sleep_s=0.4):
    ts_all = cdx_timestamps_2022_2025()
    ts_daily = choose_one_per_day(ts_all)

    frames = []
    for i, ts in enumerate(ts_daily, 1):
        try:
            html = wayback_raw_html(ts)
            date = capture_date_from_ts(ts)
            df = parse_metro_tables(html, date, ts)
            if not df.empty:
                frames.append(df)
        except Exception as e:
            print(f"Failed {ts}: {e}")

```

```

if sleep_s:
    time.sleep(sleep_s)

if i % 50 == 0:
    print(f"Processed {i}/{len(ts_daily)} days...")

if not frames:
    return pd.DataFrame()

out = pd.concat(frames, ignore_index=True)

# optional: keep only Current Avg. rows
# out = out[out["label"].str.contains("Current", na=False)].copy()

return out

if __name__ == "__main__":
    df = run_scrape()
    df.to_csv("aaa_fl_metros_wayback_2022_2025.csv", index=False)
    print(df.head())
    print("Saved aaa_fl_metros_wayback_2022_2025.csv")

```

Why this script is “correct” for your case:

- It uses CDX to discover captures
- It fetches the raw archived HTML using `id_`
- It parses exactly the structure you pasted (`h3[data-cost] → next table.table-mob`)

5) What you’ll end up with

Rows like:

- date = 2022-09-15
- metro = Fort Lauderdale
- label = Current Avg.
- regular/mid/premium/diesel = ...

That's a clean panel for synthetic control / event studies (just with metros instead of counties).

One last blunt point

If you truly need **county** and not metro, AAA's page structure you found suggests **they're not publishing county on this page**. So your next move would be:

- either accept metro-level geography, or
- switch sources for county-level (and use AAA metros only as a robustness check).

If you want, tell me which hurricane(s) you're targeting (e.g., Irma, Ian) and which metro you want as treated (Miami/Ft Lauderdale/etc.), and I'll show you how to reshape this into a proper panel for your synthetic control setup.

Success: Link copied!

Link copied!